CORELANG

Programming-Language Evolution, Constraint Modeling, Self-Modification, and Predictive Language Design

Research Dossier and Preliminary Architecture — v0.0

Research date: 13 October 2026
Status: Preliminary / non-canonical research architecture
Purpose: Execute the re-engineered Corelang research prompt in full

Central question: If programming languages are engineered systems built from rules, abstractions, execution models, historical decisions, and constraints, can those elements be represented computationally well enough that a system can systematically explore the space of possible programming languages?

Contents

1. Executive Research Summary
2. Programming-Language Census
3. Historical Evolution Analysis
4. Language Anatomy
5. Rule Taxonomy
6. Limitation Taxonomy
7. Programming-Language Evolution Graph
8. Failure and Invalid-State Model
9. Corelang Conceptual Architecture
10. Corelang Rule Representation
11. CoreIR Preliminary Architecture
12. Evolution Engine
13. Self-Model Architecture
14. Adversarial Corelang Framework
15. Stability and Invariant Analysis
16. Versioning and Reproducibility Model
17. Feasibility Matrix
18. Preliminary Corelang v0.0 Specification
19. Open Problems
20. Research Roadmap
21. Exhibits

Appendix A. Representative Language Version Matrix

Appendix B. “This Cannot Run” Diagnostic Protocol

Appendix C. Corelang Research Principles and Acceptance Gates

22. References

Scope, Method, and Evidence Standard

This dossier treats Corelang as a research program, not as a claim that a universal self-evolving language already exists. It combines current language standards and official documentation, historical programming-language databases, compiler and intermediate-representation projects, metaprogramming/language-workbench systems, program-synthesis research, fuzzing and compiler-validation techniques, formal verification, and computability theory. Current-state claims were checked against sources available on 13 October 2026. Historical lineage claims are deliberately summarized at family level when the full genealogy is too large for a single dossier.

The analysis uses a strict separation between (a) established mechanisms already deployed in programming-language ecosystems, (b) engineering integrations that are feasible but substantial, (c) active research problems, and (d) speculative or theoretically constrained ambitions. This distinction is essential because Corelang's most interesting premise—making language rules experimental objects—can be implemented in useful partial forms today, while a genuinely universal representation or unrestricted self-evolution encounters deep semantic, verification, and computability limits.

The programming-language census is intentionally expressed as a range and collection-dependent count. HOPL currently presents a roster of 8,945 languages with thousands of influence links and citations, whereas PLDB describes more than 5,000 languages and over 135,000 facts. PLDB's own discussion emphasizes that the count depends on what qualifies as a language, and previously estimated 500–2,000 active general-purpose languages and 5,000–25,000 active computer languages under broader definitions. [R1–R3]

1. Executive Research Summary

Corelang is technically most coherent when defined not as one endlessly mutable source syntax, but as a versioned language laboratory composed of a meta-language, a machine-readable language model, one or more intermediate representations, generators, sandboxes, conformance and adversarial test systems, formal and differential validators, a historical corpus, and a provenance-preserving evolution engine. The central innovation is to make a language rule—lexical, syntactic, semantic, typing, runtime, compatibility, or platform-related—a first-class object with a rationale, dependencies, alternatives, mutation operators, and test obligations.

Several pieces of this idea already exist independently. LLVM IR demonstrates a common compiler representation aimed at many high-level languages; MLIR explicitly supports multiple abstraction levels and dialects; WebAssembly separates structure, validation, and execution semantics; Racket can define new languages and reader layers; Spoofax and Rascal treat languages and programs as data for generation, analysis, and transformation; modern fuzzers evolve input corpora; Csmith showed that randomized valid-program generation can expose hundreds of compiler bugs; Test262 demonstrates large-scale implementation-agnostic conformance testing; CompCert shows that important compiler correctness properties can be machine-proved for a substantial C subset. [R22–R32]

Corelang's proposal is therefore not "invent everything from nothing." It is to integrate these ideas into a single research architecture whose subject is the design space of languages themselves. The difficult part is not generating alternative grammars. The difficult part is preserving meaning, identity, safety, reproducibility, and evidence as grammar, semantics, types, memory models, execution rules, and even the meta-rules governing those changes evolve.

The most consequential architectural recommendation is a constitutional boundary: no candidate rule set may silently rewrite the meaning of a previously recorded execution. Every experiment must be pinned to immutable identifiers for the language specification, rule graph, compiler/interpreter build, CoreIR schema, runtime, corpus snapshot, random seed, dependencies, target platform, and test suite. A future Corelang may alter the constitutional layer, but doing so must create a new constitutional generation rather than retroactively changing the old one. This converts the stability paradox from "something must never change" into "changes must be explicit, versioned, replayable, and non-retroactive."

A second conclusion is that the slogan "this cannot run" is too coarse for Corelang. The system should classify rejection at the exact layer: lexical invalidity, grammar invalidity, static semantic error, type error, capability denial, runtime trap, missing implementation, target incompatibility, resource exhaustion, security policy, undecidability, unavailable information, or physical impossibility. Only then should it ask what would have to change for the statement to become false. In many cases the answer is "change the parser, type system, runtime, or target." In other cases—such as a general solution to the halting problem—the answer is "no rule mutation removes the theoretical barrier." Computability theory therefore becomes a guardrail, not an enemy of exploration. [R34–R35]

The preliminary Corelang v0.0 specification in this dossier does not define a polished surface syntax. That would be premature. Version 0.0 instead defines the entities that must be representable: Language, LanguageVersion, Rule, GrammarProduction, SemanticOperation, TypeRelation, Effect, RuntimeCapability, PlatformAssumption, ProgramArtifact, InvalidState, Test, Counterexample, Transformation, Candidate, Experiment, EvidenceRecord, and ProvenanceRecord. Once these are stable enough, multiple human- or machine-facing syntaxes can be layered over the model.

1.1 Primary Findings

- There is no authoritative single count of programming languages; curated databases already disagree by thousands because the definition boundary is inherently fuzzy. [R1–R3]
- Programming languages evolve along multiple axes simultaneously: specification editions, implementation releases, dialects, extensions, profiles, compatibility modes, runtime versions, and library/platform versions. "How many versions?" has no single scalar answer.
- Many supposedly fixed language rules are already context- or version-sensitive. Rust editions isolate incompatible parsing changes; Kotlin distinguishes hard and soft keywords; Swift permits escaped keyword identifiers. [R13, R16, R17]
- Universal representation is useful but not perfectly lossless. LLVM IR and MLIR show broad cross-language leverage, yet source-level effects, undefined behavior, dependent types, macros, reflection, staged computation, ownership, exception models, concurrency semantics, and domain-specific invariants can be lost unless represented explicitly. [R22–R23]
- Adaptive language generation is achievable in bounded spaces today using grammars, search, synthesis, fuzzing, language workbenches, differential testing, and constraint solving. Unbounded automatic discovery of the "best possible language" is not a well-defined or computable objective without explicit goals and restrictions. [R27–R35]
- Failure is valuable data when its provenance is preserved. Invalid programs, compiler diagnostics, rejected proposals, miscompilations, runtime traps, and unsound transformations can all become counterexample corpora.
- The strongest near-term Corelang prototype is a meta-model + rule graph + multi-level IR + candidate generator + sandbox + evaluation harness, not a self-rewriting production compiler.

2. Programming-Language Census

2.1 How Many Programming Languages Are There?

The best answer is: thousands, with the exact number determined by the counting rule. HOPL identifies 8,945 programming languages "from the 18th century to the present." PLDB reports more than 5,000 programming languages and treats formats, notations, and binary specifications broadly enough to expose how ambiguous the boundary is. PLDB's historical estimate further distinguishes active general-purpose languages from all active computer languages. [R1–R3]

Table 1. Programming-language census evidence

Collection / approach	Reported scale	What the number means
HOPL	8,945	Curated historical roster/genealogy; includes obscure, experimental, and historical languages.
PLDB	5,000+	Public database of languages and related formal notations; 135,000+ facts.
PLDB estimate (2017)	500–2,000 active general-purpose; 5,000–25,000 active computer languages	Explicitly model-based ranges dependent on definition.
Standards ecosystems	Dozens of major standardized families	Counts standards and editions, not every dialect/DSL/implementation.

Corelang should therefore never store a field such as total_languages = 8945 and treat it as universal truth. It should store census observations with collection, date, inclusion criteria, exclusions, aliases, confidence, and provenance. A language census is itself a model.

2.2 What Should Count as a Language?

A useful Corelang taxonomy should distinguish at least the following categories, because collapsing them distorts both genealogy and capability analysis: general-purpose programming languages; domain-specific languages; query languages; hardware-description languages; shader languages; shell and command languages; scripting languages; visual languages; educational languages; esoteric languages; theorem/proof languages; configuration languages; build languages; serialization/data languages; markup languages; interface-definition languages; bytecode and virtual-machine instruction languages; assembly languages; machine instruction sets; intermediate representations; smart-contract languages; and embedded sublanguages.

The inclusion rule should be declarative. For example, CoreCorpus could support several census profiles: Strict-PL (languages intended to express executable computations), Formal-Computer-Language (adds declarative/query/configuration/markup systems), Execution-Representation (adds bytecodes and IRs), and Historical-Broad (adds influential formal notations and abandoned proposals). This allows the same corpus to answer different questions without pretending the category boundary is objective.

2.3 Alias, Dialect, Fork, Version, and Descendant Problems

Counts are further complicated by identity. Is Python 2 a version of Python or, after enough incompatibility, effectively a dialect? Is Raku a version of Perl 6 or a descendant language? Is a language with a new grammar but the same runtime a new language? What about SQL dialects that share a standard core but diverge sharply? Corelang should model identity as a graph, not a single parent pointer. Relationships should include renamed-from, forked-from, compatible-with, dialect-of, extension-of, transpiles-to, hosted-by, descended-from, influenced-by, and shares-runtime-with.

3. Historical Evolution Analysis

Programming-language evolution is neither purely linear nor purely genealogical. Some families move through standards (C, C++, Fortran, COBOL, Ada, SQL); some combine language and platform releases (Java, C#); some use annual or frequent implementation releases (Python, Go, Swift, Kotlin); some use explicit compatibility epochs or editions (Rust); some maintain living standards (HTML); and some have language versions coupled to ecosystem/runtime versions (Erlang/OTP, Elixir, Solidity). The correct Corelang model therefore separates Language, SpecificationEdition, ImplementationRelease, RuntimeRelease, Dialect, FeatureProposal, and CompatibilityProfile.

Table 2. Representative language/version evolution matrix

Language/family	Representative version path	Evolution lesson for Corelang
FORTRAN / Fortran	1950s → 2023 standard	From early numeric/scientific compilation through structured programming, modules, array language, object-oriented and parallel/coarray features; ISO/IEC 1539-1:2023 is the current published base-language edition. [R6]
COBOL	1959 → 2023 standard	Long-lived business language; successive standards preserve legacy compatibility while adding structured and modern facilities; ISO/IEC 1989:2023 defines syntax/semantics and explicitly notes implementation-defined/undefined elements. [R7]
Lisp family	1958 → many descendants	Lists/homoiconicity, macros, dynamic/lexical scope traditions, Common Lisp/Scheme/Racket/Clojure branches; demonstrates that syntax and language extension can themselves be programmable.
ALGOL family	1958/60/68 → descendants	Block structure, lexical scope and formal grammar strongly influenced Pascal, C and many successors; a major genealogy hub in HOPL. [R1]
C	1972 → C89/C90/C99/C11/C17/C23	Standardized evolution; ISO/IEC 9899:2024 is Edition 5 and specifies syntax, constraints, semantics, portability restrictions, while deliberately leaving compilation mechanism and platform capacity outside the language standard. [R4]
C++	1979 origins → 1998/03/11/14/17/20/23	Large accretive general-purpose language; ISO/IEC 14882:2024 is Edition 7; a subsequent draft is already under

		development, illustrating overlapping standard generations. [R5]
Ada	1983/95/2005/2012/2022 lineage	Reliability- and portability-oriented standardized language; ISO/IEC 8652:2023 specifies form, execution effect, libraries, permitted variations, and required diagnostics. [R8]
SQL	SQL-86 onward → SQL:2023	A family of multipart standards plus vendor dialects; ISO/IEC 9075-1:2023 defines the framework and grammar/processing concepts; a 2026 corrigendum and next edition work coexist. [R9]
Java	1995 → rapid six-month platform cadence	Language specification tied to Java SE/JVM specifications; Oracle lists Java SE 27 released September 2026, with prior editions retained. [R11]
ECMAScript	1997 → annual editions	ECMA-262 17th edition defines ECMAScript 2026; Edition 4 was reserved but never published, a useful example of “version numbers” not equaling implemented historical states. [R10]
Python	1991 → 2.x / 3.x → 3.15	Major semantic break from 2 to 3; annual feature release cycle from PEP 602; Python 3.15.0 released 9 Oct 2026 and 3.16.0a1 began 13 Oct 2026. [R12]
Rust	2010 → 1.0 (2015) + editions	Compiler versions advance continuously while editions 2015/2018/2021/2024 gate incompatible parsing changes. Multiple editions interoperate in one dependency graph. [R13]
Go	2009 → Go 1.x	Compatibility promise constrains evolution; Go 1.26 (Feb 2026) includes language changes while maintaining the Go 1 compatibility expectation. [R14]
Swift	2014 → 6.4.1	Language, toolchain, package manager, interoperability and platform support evolve together; Swift 6.4 arrived Sep 2026 and 6.4.1 on 13 Oct 2026. [R15]
Kotlin	2011 → 2.4.21	Language/tooling releases are explicit; 2.4.0 was a language release and 2.4.21 a bug-fix release on 8 Oct 2026. [R16]
Lua	1993 → 5.5	Small embeddable language with official manuals per release; Lua.org lists 5.5 current and manuals back through 1.0, making version-to-version comparison unusually tractable. [R17]
Erlang / OTP	1980s → OTP 28	Language evolution is coupled to the OTP runtime/platform; OTP 28 added language features such as zip generators and strict generators. [R18]
Elixir	2011 → 1.20	Demonstrates substantial semantic evolution without a new language identity: v1.20 (2026) introduced gradual type checking across programs. [R19]

Julia	2012 → 1.13	High-performance dynamic technical language; 1.13 released Sep 2026 with compiler/precompile improvements, illustrating language/runtime/performance co-evolution. [R20]
Solidity	2014 → 0.8.x	Smart-contract language where compiler semantics, EVM changes and security fixes are tightly coupled; 0.8.36 (Jul 2026) included security fixes and backend changes. [R21]
WebAssembly	1.0/2.0/3.0	A portable validated execution language/ISA with explicit validation and execution semantics; 3.0 released Sep 2026. [R24]

3.1 What Counts as an Iteration?

Corelang should track at least eight iteration dimensions: specification edition, language feature release, compiler/interpreter release, runtime/VM release, standard-library release, compatibility epoch/edition, dialect/vendor profile, and proposal/experimental feature state. Java SE 27, a particular JDK build, a JVM specification, and a preview feature are related but not identical versions. Rust compiler 1.x and Rust Edition 2024 are different axes. HTML uses a Living Standard rather than numbered annual editions. A single integer "version count" would erase this structure.

3.2 Evolution Patterns Worth Learning

- Accretion with compatibility: C++, Java, Go and many enterprise languages retain old semantics for ecosystem stability.
- Periodic compatibility epochs: Rust editions show that incompatible syntax can coexist when the version context is explicit.
- Living-standard evolution: HTML changes continuously while browser interoperability and parsing behavior anchor reality. [R25–R26]
- Feature-proposal pipelines: ECMAScript and Swift expose staged proposal processes that separate experimentation from standard adoption.
- Runtime-coupled evolution: Java/JVM, Erlang/OTP and Solidity/EVM demonstrate that a language cannot be modeled independently of its execution substrate.
- Dialect ecosystems: SQL shows that a standard language family can have large implementation-specific semantic surfaces.
- Research-to-production evolution: Elixir's gradual typing work moved from research into deployed language semantics. [R19]

4. Language Anatomy

A programming language is not merely syntax. At minimum, an executable language needs a representation of programs plus rules that determine what those programs mean. In practice, useful language systems also rely on toolchains, runtimes, libraries, package and module systems, target interfaces, and diagnostics. C's ISO standard is illustrative: it specifies representation, syntax/constraints, semantic rules, input/output representation, and implementation restrictions, while explicitly declining to define the mechanism of translation or invocation. [R4]

Table 3. Language-system anatomy

Layer	Core question	Representative artifacts
Lexical	What character sequences become tokens?	character sets, comments, literals, identifiers, keywords
Grammar / syntax	Which token structures are well formed?	EBNF/grammar productions, parser, CST/AST
Static semantics	What must be true before execution?	name binding, scope, type/effect rules, definite assignment
Dynamic semantics	What does execution mean?	evaluation rules, state transitions, exceptions/traps
Memory model	How are values/states stored and observed?	allocation, ownership, references, GC, atomics
Concurrency model	How do agents/processes interact?	threads, actors, async tasks, memory ordering
Module/linkage	How are program parts composed?	modules, packages, imports, ABI/FFI
Execution substrate	Where/how does code run?	interpreter, VM, JIT, native compiler, accelerator
Standard environment	What services are assumed?	libraries, OS APIs, browser APIs, host functions
Tooling/conformance	How is behavior validated?	test suites, linters, formatters, reference implementations

Corelang should also distinguish syntax from semantics and textual spelling from semantic role. The word return is not “the return operation”; it is one surface encoding of a control-transfer concept. This distinction is a prerequisite for generated syntax, multilingual surfaces, contextual keywords, and alternate representations of one program.

4.1 When Does a Notation Become a Programming Language?

There is no universally enforced boundary. A practical Corelang classification can use capability dimensions instead of a binary label: can the notation express conditional behavior, repetition/recursion, state transformation, abstraction, composition, effects, interaction with a host, or general computation? SQL, HTML, CSS, regexes, build systems, proof languages and configuration DSLs each occupy different points. The system should record intended purpose and computational properties rather than forcing every artifact into “programming language” or “not language.”

5. Rule Taxonomy

Corelang’s rule model should begin by classifying what is being constrained and why. A rule should not be considered immutable merely because a compiler currently enforces it. Conversely, a rule should not be considered safely mutable merely because it is conventional. The following taxonomy makes the rationale explicit.

Table 4. Corelang rule taxonomy

Rule class	What it constrains	Typical rationale
Lexical	valid token formation; identifier shape;	Usually design + tooling; sometimes

	comment/literal rules	interoperability/security
Syntactic	grammar productions and precedence	Design, ambiguity control, parsing strategy
Name/binding	scope, visibility, namespace resolution	Semantic coherence; modularity
Type	which values/operations compose	Safety, proof power, optimization, documentation
Effect/capability	which operations may access state/resources	Safety, isolation, host policy
Memory	lifetime, aliasing, ownership, GC, layout constraints	Safety, performance, hardware/runtime model
Concurrency	synchronization, ordering, message semantics	Correctness, determinism, hardware memory model
Runtime	traps, exceptions, dynamic checks	Execution semantics, recovery
Platform	target ABI, instruction support, host APIs	External dependency
Compatibility	deprecated/legacy behavior, editions	Ecosystem stability
Security	sandbox, permissions, unsafe operations	Threat model / isolation
Reserved-word	identifier exclusions/contextual tokens	Parser disambiguation and readability; often conventional
Undefined/unspecified	implementation freedom or intentionally unmodeled behavior	Optimization latitude, hardware variation, historical compromise
Resource	limits on memory/time/stack/code size	Implementation/physical capacity, not usually language semantics

5.1 Reserved Words Are Already More Flexible Than They Look

The user's reserved-word example is particularly tractable. Rust editions allow new keywords to become meaningful only in newer editions while old crates continue to compile. Kotlin distinguishes hard keywords from soft keywords that may act as identifiers outside specific contexts. Swift permits many keyword-shaped identifiers when escaped with backticks and has moved some historically reserved number-sign forms into macros. [R13, R16, R17] Corelang therefore does not need to "break language theory" to make reserved vocabulary adaptive; it needs a precise lexer/parser context model, symbol-role separation, migration tooling, and version-pinned grammar.

6. Limitation Taxonomy

The phrase "programming languages cannot do X" is ambiguous. A language can fail to express an operation, a compiler can fail to implement it, the runtime can lack a capability, the operating system can forbid it, the hardware can lack an instruction, or the requested property can be undecidable. Corelang must identify the layer before proposing a mutation.

Table 5. Limitation classification

Limitation class	Diagnosis	Corelang response
Language-specific	No syntax/semantic form exists	Add syntax/primitive/library/FFI; or translate to another representation

Compiler	Specification permits it but implementation does not	Implement frontend/middle-end/backend support
Runtime	Needed service/value model absent	Extend VM/runtime or host interface
OS/host	Permission/API unavailable	Change host capability or execution environment
Hardware	Target lacks required facility	Emulate, lower differently, or select another target
Security policy	Operation intentionally denied	Require explicit capability; keep denial if threat model demands it
Resource	Too much time/memory/energy/storage	Optimize, approximate, distribute, or reject
Information	Required facts are unavailable	Acquire data, weaken objective, or declare unknowable
Decidability/computability	No general algorithm exists	Restrict domain; use semi-decision, approximation, proof obligations; cannot “syntax” away theory
Physical	Violates physical constraints	No software-language change can remove the constraint
Compatibility	Would break existing programs/contracts	Use edition/profile/migration or fork
Historical/conventional	Rule persists by habit/design choice	Candidate for experiment if dependencies are understood

Computability theory is a hard boundary. The halting problem and broad classes of semantic program properties are undecidable in the general case. Corelang can improve analyzers for restricted domains, require annotations, bound execution, use abstract interpretation, run tests, or search for proofs, but it cannot create a general algorithm that correctly decides every nontrivial semantic property of arbitrary programs. [R34–R35]

7. Programming-Language Evolution Graph

The full historical graph is too large to reproduce meaningfully in a static document—HOPL alone reports thousands of influence links—but Corelang should ingest it as graph data rather than compress it into a family tree. [R1] A graph permits multiple parents, cross-paradigm influence, shared runtimes, standardization forks, and feature migration.

7.1 Proposed Node Types

- Language
- Version / edition
- Feature
- Rule
- Grammar production
- Compiler/interpreter
- Runtime/VM
- Standard
- Proposal

- Library/host platform
- Paradigm
- Error/failure
- Test/counterexample
- Hardware/ISA
- Paper/specification/author

7.2 Proposed Edge Types

- influenced-by
- descended-from
- forked-from
- renamed-from
- dialect-of
- implements
- compiled-to
- hosted-by
- standardized-as
- introduced-feature
- deprecated-feature
- removed-feature
- incompatible-with
- requires-runtime
- violates-rule
- fixes-failure
- tested-by
- semantically-equivalent-under-profile
- lowered-to
- cannot-losslessly-lower-to

7.3 Transferable Innovation Examples

Some of the most useful innovation paths are cross-family: lexical scope from ALGOL-family designs; garbage collection and macros from Lisp traditions; object systems from Smalltalk and successors; algebraic data types and pattern matching from ML/Haskell families; actors from Erlang; ownership/borrowing from Rust; gradual typing across dynamic languages; async/await across multiple ecosystems; generics/templates across C++, Java, C#, Rust and others; and WebAssembly-style validation/embedding boundaries. Corelang should encode the feature as its own node and connect each language-specific realization rather than treating similarly named features as identical.

8. Failure and Invalid-State Model

CoreFailure is one of the strongest differentiators in the proposed architecture. Traditional tooling often treats invalid input as disposable. A language-research system should preserve it as a counterexample with exact provenance. Compiler testing supports this philosophy: Csmith generated valid random C programs while avoiding undefined/unspecified behavior and reported more than 325 previously unknown compiler bugs; libFuzzer retains inputs that discover new coverage and saves crash/timeout artifacts; Test262 maintains tens of thousands of conformance tests covering observable specification behavior. [R29, R31, R36]

8.1 Invalid-State Record

Table 6. Proposed CoreFailure record

Field	Meaning
artifact_id	Hash-addressed source/IR/binary/test input
language_profile	Exact language/version/dialect/feature flags
rule_set	Rule graph hash and constitutional generation
toolchain	Compiler/interpreter/runtime IDs and builds
environment	OS/architecture/host capabilities/resource limits
classification	lexical/syntax/type/static/runtime/UB/security/platform/ resource/etc.
diagnostic	Machine-readable diagnostic plus human rendering
minimal_reproducer	Reduced counterexample if available
expected_behavior	Specification-derived oracle or metamorphic relation
observed_behavior	Output/trap/crash/performance/proof result
seed	Random/generative seed and generator version
lineage	Parent mutations / transformations
status	open/reproduced/fixed/expected/invalid-test
provenance	Source citations, timestamps, signatures/hashes

8.2 Failed Language Ideas

The same principle applies above the program level. A rejected syntax, abandoned proposal, unsuccessful type rule, slow runtime strategy, or language that lost adoption for ecosystem reasons may still contain a useful design. Popularity is therefore evidence about adoption, not proof of architecture. Corelang should preserve candidate variants with failure causes such as incoherent semantics, unsoundness, unacceptable migration cost, poor performance, inadequate tooling, confusing human factors, or simply external market failure.

9. Corelang Conceptual Architecture

The recommended architecture is layered and explicitly separates evidence, representation, generation, execution, evaluation, and governance. See Exhibit 1.

Table 7. Proposed Corelang subsystem decomposition

Component	Responsibility
CoreConstitution	Defines identity, provenance, isolation, reproducibility, approval gates, and how rule sets themselves may change.
CoreCorpus	Specifications, manuals, grammars, source examples, test suites, papers, implementations, historical artifacts.
CoreHistory	Version/dialect/proposal genealogies and influence graph.
CoreFailure	Invalid states, counterexamples, diagnostics, rejected proposals, miscompilations, unsound transformations.

CoreModel	Language metamodel: syntax, semantics, types, effects, memory, runtime and platform assumptions.
CoreRules	First-class rule objects and dependency graph.
CoreIR	Multi-level representation from source constructs through progressively lower semantic/target forms.
CoreGen	Generates candidate syntax/rules/types/semantics/runtime configurations and migration transforms.
CoreTest	Conformance, property, differential, metamorphic, fuzz, benchmark and regression execution.
CoreAdversary	Generates counterexamples and competing architectures designed to expose weaknesses.
CoreEvolution	Scores, retains, rejects, mutates and recombines candidates under explicit objective functions.
CorePredict	Forecasts probable language evolution and searches unexplored design regions; never treated as a correctness oracle.

This decomposition also answers an important naming question: Corelang is probably the umbrella system, not merely the human-facing syntax. A future surface language could still be called Corelang, but the research architecture needs separable components so that the model can evolve without forcing every subsystem into a single compiler binary.

10. Corelang Rule Representation

Rules should be immutable records inside a given rule-set snapshot, even though new snapshots may replace them. “Mutable rule” should therefore mean versioned replacement, not in-place historical mutation. This preserves reproducibility while still permitting evolution. See Exhibit 3.

Table 8. CoreRules preliminary schema

Rule field	Purpose
rule_id	Stable identity across revisions
revision_id	Immutable content hash/version
category	lexical/syntax/type/semantic/runtime/etc.
scope	language/version/profile/target/context
formal_statement	Machine-readable rule or executable predicate where feasible
human_statement	Human explanation
rationale	Why the rule exists
provenance	Standard/paper/implementation/historical source
dependencies	Rules and assumptions required
conflicts	Known incompatible rules
failure_class	How violations manifest
security_class	Threat/safety implications

compatibility_cost	Expected breakage/migration burden
mutation_operators	Permitted transformations
test_obligations	Required conformance/property/adversarial tests
proof_obligations	Formal properties required when applicable
status	active/experimental/deprecated/rejected
supersedes	Prior rule revision(s)

10.1 Example: Reserved Keyword Mutation

Suppose Rule K-104 states that token “await” is reserved in grammar context C. A Corelang experiment does not merely delete K-104. It produces candidate K-104b: “await is contextual; lexical classification depends on production P and edition E.” The experiment generates a migration set, parses legacy corpora, checks ambiguities, compiles representative programs, runs conformance tests, verifies symbol resolution, checks IDE tooling, and records collisions. If the candidate passes a profile-specific acceptance threshold, it becomes part of a new rule-set snapshot. Rust editions, Kotlin soft keywords, and Swift escaping demonstrate real-world fragments of this strategy. [R13, R16, R17]

11. CoreIR Preliminary Architecture

No single low-level IR can preserve every relevant property of every source language. LLVM IR is intentionally low-level and SSA-based; MLIR addresses the “semantic gap” by supporting multiple levels and dialects; WebAssembly defines a typed validated instruction language with explicit execution semantics. CoreIR should adopt a multi-level approach rather than searching for one universal AST. [R22–R24]

11.1 Recommended Representation Stack

Table 9. Proposed multi-level CoreIR

Level	Purpose
CoreIR-Surface	Lossless source/CST layer: tokens, comments, source mapping, macros, syntactic sugar, dialect/profile.
CoreIR-Semantic	Resolved symbols, typed/effect-annotated operations, ownership/lifetime, exceptions, contracts, capabilities.
CoreIR-Behavior	Control/data/effect graphs and abstract state transitions; intended for analysis and transformations.
CoreIR-Portable	Lowered portable operations with explicit memory/concurrency/host assumptions.
CoreIR-Target	Target-specific machine/VM/accelerator operations and ABI details.

The important design rule is “do not lower away information before the experiment that needs it.” A Rust borrow relationship, a Haskell laziness property, a Prolog search semantics, a SQL null/three-valued logic rule, or a JavaScript coercion behavior may disappear if translated too early into a generic

imperative CFG. CoreIR therefore needs dialect-like extension points, explicit semantic attributes, and provenance links back to source rules.

11.2 What Universality Can and Cannot Mean

A realistic universality claim is representational extensibility: CoreIR can add dialects/schema extensions capable of carrying language-specific semantics and can define verified or tested lowerings where equivalence is meaningful. An unrealistic claim is that every language can be translated into one fixed minimal IR with all semantics, metaprogramming behavior, tooling intent, performance contracts, and source-level identity preserved automatically. Even LLVM's documentation describes its goal as representing "all" high-level languages cleanly in the context of its compilation strategy, while MLIR exists precisely because different abstraction levels matter. [R22–R23]

12. Evolution Engine

CoreEvolution should be an experiment orchestrator, not an autonomous authority that can promote any self-generated rule. Its basic cycle is Observe → Normalize → Model → Generate → Execute → Attack → Verify → Score → Retain/Reject → Archive, then repeat (Exhibit 2).

12.1 Candidate Generation

Candidate generation can combine deterministic transforms, grammar mutation, constraint solving, enumerative program synthesis, stochastic search, evolutionary algorithms, LLM-assisted proposal generation, and feature recombination. Program synthesis research already frames synthesis as finding a program satisfying a specification and identifies enumerative, constraint-based, stochastic, and deductive families of methods. Corelang generalizes the target from "program" to "language component," but the search-space explosion becomes even more severe. [R28]

12.2 Evaluation Dimensions

- syntactic determinacy / parser behavior
- semantic coherence
- type/effect soundness where claimed
- memory and concurrency safety properties
- conformance to declared profile
- expressiveness on target benchmark suites
- implementation complexity
- compile/startup/runtime performance
- resource use
- portability
- interoperability
- migration cost
- debuggability and diagnostic quality
- human learnability/readability studies
- tooling feasibility
- formal-verification tractability
- security/capability confinement
- reproducibility

There is no single scalar “fitness” without policy. A language optimized for verified avionics, shader compilation, data analysis, embedded systems, theorem proving, browser applications, and beginner education will legitimately have different objective functions. Corelang therefore needs named evaluation profiles and Pareto frontiers rather than one global score.

12.3 Testing Strategy

The minimum test stack should combine: parser/grammar tests; conformance suites; property-based tests; fuzzing; randomized valid-program generation; differential execution across independent implementations; metamorphic relations; translation validation; static analyses; formal proofs for selected invariants; regression corpus replay; performance and resource benchmarks; and adversarial security tests. LibFuzzer, Csmith, Test262, Alive2, and CompCert demonstrate complementary parts of this stack. [R29–R32, R36–R37]

13. Self-Model Architecture

“Corelang models itself” should have a precise meaning. The self-model is not consciousness and not a free-form narrative. It is an executable/queriable inventory of the exact language architecture currently in force: grammar, rule graph, semantics, compiler passes, runtime contracts, supported targets, tests, known failures, generated candidates, dependencies, and constitutional constraints.

Table 10. Self-model concepts that must not be conflated

Concept	Meaning in this dossier
Introspection	Read current compiler/runtime/language metadata.
Reflection	Inspect and sometimes manipulate program structure/types at runtime or compile time.
Metaprogramming	Treat programs or language structures as data; generate/transform code.
Self-hosting	Implement the language compiler/interpreter substantially in the language itself.
Bootstrapping	Use an existing implementation to create a newer/native/self-hosted implementation.
Program synthesis	Automatically search for programs satisfying a specification.
Self-modifying code	Runtime program instructions alter code/state representing code.
Architectural self-model	Explicit machine-readable model of language rules, implementation pipeline, dependencies, evidence and change candidates.

Rascal’s description of metaprogramming—programs as data—and Racket’s explicit facilities for defining reader and expander layers show that a substantial portion of self-description is ordinary computer science when properly scoped. The ambitious step is closing the loop between description, candidate generation, verification, and governed adoption. [R26–R27]

14. Adversarial Corelang Framework

Adversarial Corelang variants should be internal design competitors, not agents authorized to bypass external systems. Their job is to produce counterexamples, alternative semantics, stress cases, and incompatible design pressures. This is closely aligned with differential testing and adversarial verification.

Table 11. Initial adversarial Corelang variants

Variant	Design pressure
A — Proof-first	Strong static types/effects, deterministic semantics, restricted metaprogramming, high proof burden.
B — Dynamic-first	Late binding, runtime adaptation, structural types, reflection, rapid DSL construction.
C — Systems-first	Explicit ownership/capabilities, predictable resource use, low-level control.
D — Language-oriented	Macros, grammar extension, domain-specific sublanguages, staged compilation.
E — Counterexample engine	Searches specifically for ambiguities, unsoundness, divergence, optimizer mismatches and migration failures.
F — Synthesizer	Attempts to recombine successful properties from A–D while E attacks the result.

The system should preserve dissent. If A proves a property under assumptions that B rejects, Corelang should store the assumptions rather than flatten the dispute into an average score. A candidate may be superior under one profile and inferior under another.

15. Stability and Invariant Analysis

The “if every rule can change, is there still a language?” problem is genuine. A language whose grammar and semantics mutate in place without version identity becomes impossible to reason about. Corelang therefore needs invariants at the level of historical identity even if no semantic feature is eternally privileged.

15.1 Recommended Constitutional Invariants

- No retroactive semantic mutation: an existing specification snapshot is immutable once published.
- Every execution and test result is attributable to a complete environment/rule-set manifest.
- Every transformation has explicit source and target models plus declared preservation claims.
- Every promotion from experimental to supported status has machine-readable evidence and approval policy.
- Generated code/rules execute only inside a declared capability and resource sandbox until promoted.
- Provenance is append-only; failures are not erased when later versions succeed.
- Unknown/undecided properties remain labeled unknown rather than coerced into pass/fail.
- The system distinguishes semantic equivalence, observational equivalence under a profile, and mere test agreement.

- Constitutional changes create a new constitutional generation and a migration relationship to the old generation.

15.2 Recursive Governance

If the constitutional layer can itself change, what governs that change? The proposed answer is not an infinitely recursive meta-language. It is an externalized transition protocol: generation G defines the admissible transformation and evidence needed to instantiate generation G+1; once G+1 exists, it does not rewrite G. The ultimate base is then not an eternal semantic rule but an evidence-preserving history of transitions implemented by ordinary computing infrastructure. Some trust remains external—hardware, operating systems, cryptographic hashes/signatures, proof checkers, and human governance—and Corelang should model these as trust assumptions rather than pretending to eliminate them.

16. Versioning and Reproducibility Model

Corelang should use a version graph, not just semantic-version strings. A program artifact carries a Language Manifest identifying the exact interpretation context. This is analogous to Rust's edition pinning, but generalized across grammar, semantics, runtime, and toolchain. [R13]

Table 12. Corelang reproducibility manifest

Manifest field	Pinned state
constitution	constitutional generation/hash
language_spec	Corelang rule-set hash
surface_profile	grammar/syntax profile + keyword map
coreir_schema	IR version + dialect set
compiler	frontend/middle/backend commit/build
runtime	runtime/VM build + capability profile
stdlib	library set and package lock
target	ISA/OS/ABI/browser/host profile
features	experimental/stable feature gates
corpus	test/evidence corpus snapshot
randomness	generator algorithms and explicit seeds
resources	time/memory/threads/network/filesystem limits
provenance	signatures/hashes + parent generation

This allows the language to evolve aggressively without making historical programs meaningless. A source file may target a named profile ("Corelang 0.7 / Systems-2029") or an immutable manifest hash. Migration tools can produce a new artifact plus a transformation report, not silently reinterpret old code.

17. Feasibility Matrix

Table 13. Corelang feasibility matrix

Capability	Classification	Rationale
Machine-readable grammar/rule corpus	AVAILABLE NOW	Existing parser generators, standards grammars, language workbenches and schema systems support this directly; ingestion remains labor-intensive.
Historical language knowledge graph	ACHIEVABLE WITH ENGINEERING	HOPL/PLDB-like data plus standards and repositories can be normalized into graph entities, but identity resolution is substantial.
Cross-language multi-level IR	ACHIEVABLE WITH ENGINEERING	LLVM/MLIR demonstrate the pattern; CoreIR can be extensible, though not losslessly universal. [R22–R23]
Generated syntax and DSL variants	AVAILABLE NOW	Racket/Spoofax-style language construction and parser generators already support this. [R26–R27]
Generated type/effect rule candidates	ACTIVE RESEARCH PROBLEM	Possible in constrained domains using synthesis/SMT/proof systems; difficult to guarantee soundness and usability at scale.
Automated candidate compiler generation	ACHIEVABLE WITH ENGINEERING	Language workbenches can generate parsers/typecheckers/transforms for declarative subsets; complete high-performance compiler generation is much harder.
Invalid-state/counterexample corpus	AVAILABLE NOW	Fuzzers, conformance suites and compiler-testing pipelines already preserve artifacts. [R29, R31, R36]
Differential/adversarial compiler testing	AVAILABLE NOW	Csmith, Test262 and multi-implementation differential testing provide established approaches. [R31, R36]
Translation validation of selected transforms	AVAILABLE NOW	Alive2 demonstrates automatic verification for many LLVM-level transformations, with stated scope limits. [R37]
Formally verified compiler chain for a bounded language	AVAILABLE NOW	CompCert demonstrates a realistic production-quality verified compiler for a substantial C subset. [R30]
Automatic language evolution under explicit benchmarks	THEORETICALLY PLAUSIBLE	Search/evolution is implementable, but benchmark choice determines what “improvement” means and search cost can explode.
Prediction of likely future language features	SPECULATIVE	Historical/market/model signals can support probabilistic forecasts; no guarantee that predictions identify superior designs.
Automatic invention of useful new abstractions	ACTIVE RESEARCH PROBLEM	Generative models and synthesis can propose candidates, but novelty, semantics, correctness and human utility require validation.
Perfect translation among all languages with zero semantic loss	FUNDAMENTALLY CONSTRAINED	Different languages expose incompatible semantics and undefined/implementation-defined behavior; a fixed universal lowering cannot preserve all source distinctions.

General decision procedure for whether any arbitrary program halts	IMPOSSIBLE UNDER CURRENT THEORY	The halting problem is undecidable. [R34]
General perfect detector of every nontrivial semantic program property	IMPOSSIBLE UNDER CURRENT THEORY	Rice-style undecidability blocks a universal algorithm; restricted analyzers remain useful. [R35]
Language rule changes that override physical resource limits	FUNDAMENTALLY CONSTRAINED	Representation changes can trade resources or use other hardware; they cannot repeal physical constraints.
Production self-modification with no stable provenance/version identity	SPECULATIVE / NOT RECOMMENDED	Technically code can modify itself, but unversioned semantic mutation destroys reproducibility and assurance.
Evolving constitutional layer via explicit generations	ACHIEVABLE WITH ENGINEERING	Treat each constitutional generation as immutable and define transition evidence rather than in-place mutation.

18. Preliminary Corelang v0.0 Specification

Corelang v0.0 is a research specification for representing and testing language designs. It is intentionally not a final end-user language. The v0.0 objective is to make language assumptions explicit enough to compare, mutate, execute, and evaluate them reproducibly.

18.1 Core Entities

Table 14. Corelang v0.0 entity model

Entity	v0.0 role
Language	Identity graph node for a language family/concept.
LanguageVersion	Immutable profile of a language at a point in standards/implementation history.
Rule	First-class lexical/syntactic/semantic/type/runtime/etc. constraint.
GrammarProduction	Structured syntax rule with precedence/context metadata.
SemanticOperation	Meaning independent of a particular token spelling.
TypeRelation	Subtyping, compatibility, inference, ownership/effect relation.
Effect	State/I/O/concurrency/capability behavior.
RuntimeCapability	Host resource/action that code may request.
PlatformAssumption	OS/ISA/ABI/browser/VM/hardware premise.
ProgramArtifact	Source/AST/IR/binary plus manifest.
InvalidState	Rejected or failing construction with classification.
Transformation	Source→target mapping with preservation claim.
Candidate	Generated rule/language architecture under experiment.
Test	Input + oracle/property/metamorphic relation.
Counterexample	Artifact that falsifies a claim or breaks an implementation.
Experiment	Fully specified execution of candidates/tests.
EvidenceRecord	Results, proofs, benchmarks and uncertainty.

ProvenanceRecord	Origin, citations, hashes, lineage and timestamps.

18.2 Minimal Rule DSL (Illustrative, Not Canonical Syntax)

A future Corelang rule DSL could express declarations conceptually like the following. This is a data model example, not a final syntax:

```
rule K-104 revision b7f2...
category lexical.contextual-keyword
scope profile Corelang.Experimental.4
semantic_role AwaitOperation
surface_token "await"
active_when grammar Production.AsyncExpression
legacy_collision identifier("await")
mutation_of K-104a
obligations [parse-unambiguous, migration-complete, tests:await-suite, differential:2-implementations]
status experimental
```

18.3 Variable Is Not a Fundamental Container

The familiar “variable is a container” description is pedagogical, not a law of computation. Languages already model bindings, references, immutable names, streams, futures/promises, generators, lazy thunks, capabilities, symbolic terms, logic variables, reactive signals, distributed references, linear resources, borrowed views, continuations, and values whose representation is compiler/runtime dependent. Corelang should therefore represent binding semantics independently from storage semantics.

The requested “object that cannot be contained” can be operationalized as a research family rather than a mystical primitive. Candidates include an unmaterialized stream (only the next portion exists), a capability (authority relation rather than stored payload), a remote resource identity, a symbolic constraint denoting a set of possible values, a temporal/event-defined value, an affine/linear resource that cannot be duplicated, a continuation, or a context-indexed computation. The experiment is to define the semantics and observations precisely, then determine which existing language concepts already cover them and where genuinely new behavior remains.

18.4 Dynamic Vocabulary

Corelang v0.0 should separate SemanticOperationID from SurfaceToken. A profile may bind one operation to multiple spellings or assign a spelling contextually. This enables temporary DSL vocabulary, generated operators, multilingual surfaces, and edition-specific tokens while retaining a stable semantic reference. Ambiguity is resolved by scoped grammar and precedence rules, not by assuming one global forever-reserved word table.

18.5 Web and Multi-Language Systems

HTML is a useful case because its parser behavior is defined independently from older SGML assumptions and the Web platform composes HTML syntax, DOM structures, CSS, JavaScript, WebAssembly, network protocols and host APIs. The WHATWG standard explicitly distinguishes authoring syntax from parsing rules, and WebAssembly separates core semantics from embedder APIs.

[R24–R26] Corelang should therefore model a “language environment” as a composition of cooperating languages and hosts, not insist that one language own the entire computation.

18.6 Language Models

LLMs can contribute candidate generation, documentation mining, translation suggestions, test generation and pattern discovery, but probability must never be equated with formal validity. Corelang’s pipeline should maintain separate statuses: generated → parsed → statically valid → type/effect valid → executable → tests passed → differential agreement → property verified → formally proved (for a stated model). An LLM can propose; the rule/test/proof infrastructure decides what claims the proposal earns.

18.7 Randomization Case Study

Randomization is an ideal demonstration that “the algorithm ran” is not the same as “the property is satisfied.” NIST separates deterministic random-bit generator mechanisms, entropy sources, and constructions combining them; predictable or repeated seeds can create security weaknesses. [R33] Corelang should store generator algorithm, seed, entropy assumptions, statistical objectives and security profile as separate metadata. The mnemonic labels “A IDE” and “A DIE” may remain project markers only until formally defined; they should not be assigned technical meaning by inference.

19. Open Problems

Table 15. Corelang open-problem register

Open problem	Research question
Semantic equivalence	How should Corelang state equivalence across languages with different observable behaviors, UB, concurrency, I/O, reflection, or timing?
Universal representation	Where should CoreIR use shared operations versus dialect-specific semantics? When is lowering provably or observationally lossless?
Search-space explosion	The number of possible grammar/type/runtime combinations is enormous; what priors, constraints, synthesis methods and objectives make search tractable?
Fitness specification	Who defines “better”? Performance, safety, brevity, proofability, readability and compatibility often conflict.
Human factors	A machine-generated language may be internally elegant yet impossible for humans to learn/debug. How are usability experiments incorporated?
Proof scalability	Formal verification is powerful but costly; which properties and layers deserve proofs versus tests or translation validation?
Bootstrap trust	Even verified components rely on proof checkers, compilers, hardware and build systems. How is the trusted computing base minimized and audited?
Self-reference	How much of Corelang can safely model and transform itself before circular dependencies make evidence uninterpretable?
Backward compatibility	Can rule mutation automatically produce migration transforms with correctness claims strong enough for large

	ecosystems?
Data rights and licensing	A corpus spanning specifications, code, tests and historical artifacts has copyright/license/provenance obligations.
Security of generated semantics	Novel execution features may create new classes of capability leakage, side channels or unsafe memory behavior.
Undefined behavior	Should Corelang preserve UB as a deliberate optimization contract, eliminate it, or make it profile-specific? Different target domains may choose differently.
Concurrency	Equivalent sequential behavior does not imply equivalent concurrent behavior; memory models must be first-class.
Non-determinism	Reproducibility and non-deterministic semantics must coexist via controlled seeds, traces, schedules, and replay models.
Predictive claims	Historical trends can predict adoption patterns, but they do not prove what the next technically superior language should be.

20. Research Roadmap

Table 16. Corelang staged research roadmap

Phase	Order	Work	Exit artifact
Phase 0 — Governance & schema	0–1	Freeze terms, provenance model, constitutional generation 0, research licenses, entity/rule schemas.	Machine-readable schemas; acceptance gates; corpus policy.
Phase 1 — Historical corpus	1–2	Ingest 50–100 representative languages/versions, HOPL/PLDB links, official grammars/specs, version relationships.	CoreCorpus v0.1; CoreHistory graph; census profiles.
Phase 2 — Rule extraction	2–3	Encode lexical/grammar/type/runtime rules for 10–15 languages and their changes across versions.	CoreRules v0.1; rule rationale/dependency graph.
Phase 3 — CoreIR prototype	3–5	Build Surface/Semantic/Behavior layers for a constrained set (e.g., C subset, Python subset, small functional DSL, Wasm).	Round-trip representation; explicit loss reports.
Phase 4 — Failure corpus	4–5	Import conformance tests, compiler diagnostics, fuzz artifacts, historical rejected cases.	CoreFailure v0.1; reducers; counterexample schema.
Phase 5 — Generator/test lab	5–7	Generate grammar/keyword/type-rule variants; compile/interpret in sandbox; differential/fuzz/property testing.	CoreGen/CoreTest prototype; experiment manifests.
Phase 6 — Adversarial variants	7–8	Instantiate A–F competing profiles; automated counterexample generation	CoreAdversary v0.1; scored candidate archive.

		and Pareto scoring.	
Phase 7 — Formal layer	8–10	Select small semantics and transformations for mechanized proof/translation validation.	Proof obligations; trusted-base report.
Phase 8 — Self-hosting path	10+	Implement portions of Corelang tooling in a stable Corelang profile; do not self-host before semantics/provenance are mature.	Bootstrap chain; reproducible builds; independent validators.
Phase 9 — Predictive research	parallel	Train/fit models on historical feature/version graph; separate forecasting from normative design search.	Forecast benchmarks; calibrated uncertainty; design-space explorer.

20.1 Recommended First Prototype

The first prototype should deliberately avoid “write a whole new language.” A better proof-of-concept is a Rule Mutation Laboratory supporting three small source languages or subsets plus WebAssembly as one execution target. It should ingest grammar + semantic rules, produce a normalized rule graph, mutate one controlled dimension (for example contextual keywords, numeric overflow semantics, or pattern-matching syntax), generate migration transforms, compile/interprete test programs, run conformance and differential tests, and emit a signed experiment report. If this cannot be made reproducible and interpretable, larger self-evolution claims are premature.

21. Exhibits

The following figures are conceptual architecture exhibits. They are not implementation diagrams and should be revised as the v0.0 schemas become executable.

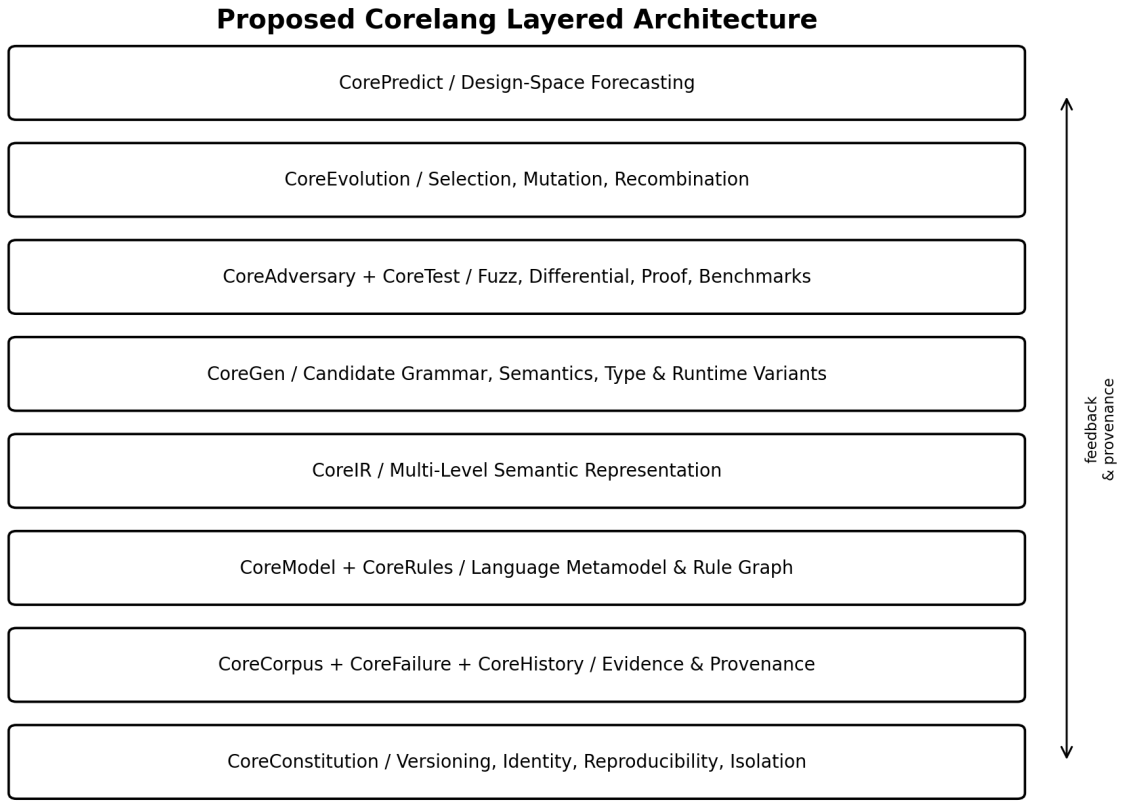


Exhibit 1. Proposed layered Corelang architecture. Evidence and constitutional provenance form the base; generation/evolution/prediction operate above explicit models and test systems.

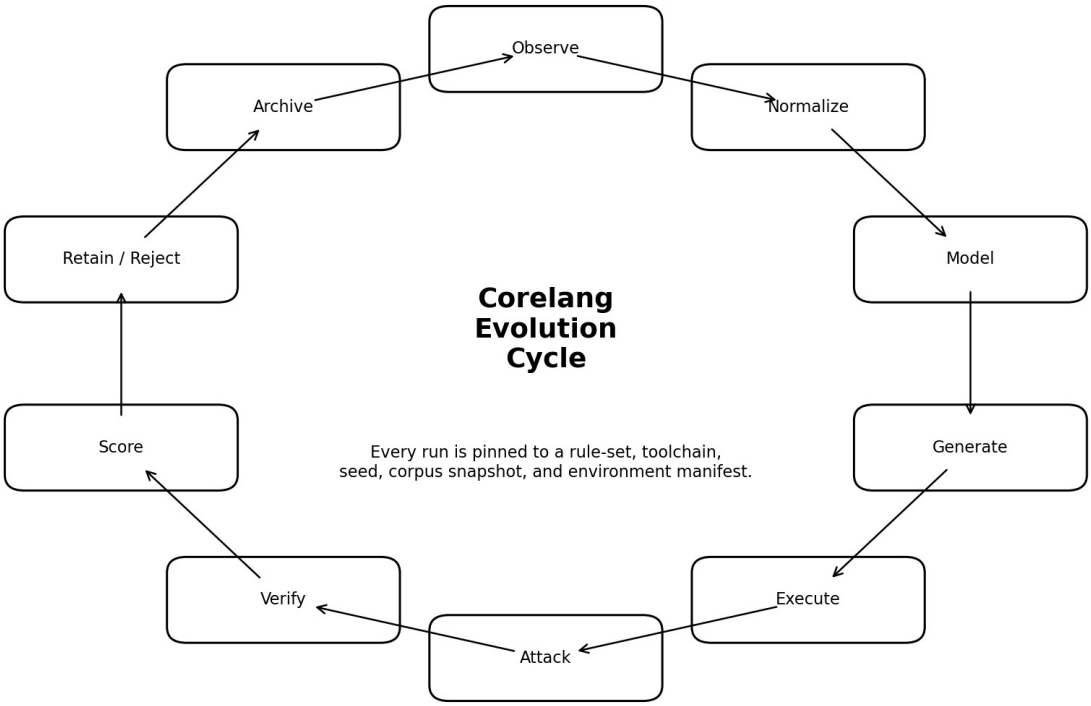


Exhibit 2. Corelang evolution cycle. Every iteration is archived with explicit manifests; “retain/reject” is evidence-based and reversible in the historical record.

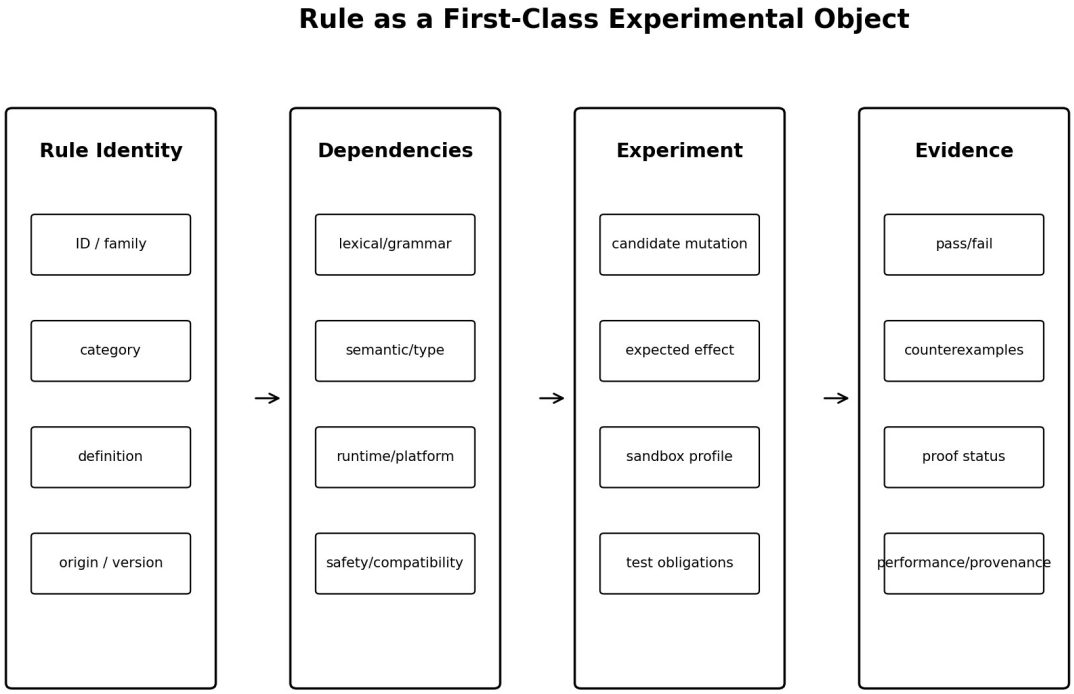


Exhibit 3. Rule as a first-class experimental object. A rule links identity and rationale to dependencies, candidate mutations, and evidence obligations.

Appendix A. Representative Language Version Matrix

This appendix distinguishes language versioning mechanisms rather than attempting to enumerate every historical release. “Current” means the most recent state verified from the cited official sources for this research date where such a source was queried.

Table A1. Representative versioning models

Family	Versioning model	Verified representative state	Corelang modeling note
C	ISO editions	C23 / ISO/IEC 9899:2024	Standards define syntax, constraints, semantics and implementation limits; toolchain versions separate.
C++	ISO editions + compiler modes	ISO/IEC 14882:2024; next edition draft underway	Feature adoption often precedes/follows formal edition depending on compiler.
Fortran	ISO editions/corrigenda	ISO/IEC 1539-1:2023 + 2026 corrigendum	Stable scientific lineage with periodic standards.
COBOL	ISO editions + vendor dialects	ISO/IEC 1989:2023	Standard plus major long-lived implementation dialects.
Ada	ISO editions	ISO/IEC 8652:2023	Edition 4; successor work item underway.
SQL	Multipart ISO editions + vendor dialects	SQL:2023 framework + 2026 corrigendum	Standard is not identical to PostgreSQL/Oracle/SQL Server/MySQL dialects.
ECMAScript	Annual ECMA editions + living draft	ECMAScript 2026 / ECMA-262 17th	Edition 4 reserved but not published; proposal stages feed next edition.
Java	Java SE editions + JDK builds + JVM spec	Java SE 27 (Sep 2026)	Language and VM specifications co-evolve; preview features add another state dimension.
Python	Feature + maintenance releases	3.15.0 current stable feature release; 3.16.0a1 preview	Annual feature cadence; alpha/beta/RC/maintenance/security states.
Rust	Compiler releases + editions	2015/2018/2021/2024 editions	Edition isolates selected incompatible parsing changes while ecosystem can mix editions.
Go	Go 1.x releases	1.26	Strong compatibility promise; implementation/runtime/library and language changes bundled.
Swift	Toolchain/language releases	6.4.1 on research date	Fast-moving language plus platform/toolchain interoperability.
Kotlin	Language/tooling/bugfix releases	2.4.21	Release metadata explicitly distinguishes language vs tooling/bugfix releases.
Lua	Language releases with official manuals	5.5	Compact sequence with manuals back to 1.0.
Erlang	OTP major/patch releases	OTP 28.x	Language features coupled to runtime/platform release.
Elixir	Language releases on BEAM	1.20	Gradual typing milestone demonstrates large semantic evolution within same family.
Julia	Language/runtime releases	1.13	Compiler/runtime performance

			and language capability evolve together.
Solidity	Compiler/language 0.x releases	0.8.36 verified; docs already track later development	Security fixes and backend/EVM assumptions make exact compiler version critical.
WebAssembly	Core spec releases + proposals	3.0	Validation/execution semantics plus embedder specs and feature proposals.
HTML	Living Standard	Continuous	Snapshot count is less meaningful than exact spec commit/date + browser implementation behavior.

Appendix B. “This Cannot Run” Diagnostic Protocol

Table B1. Corelang impossibility-challenge protocol

Step	Action
1. Parse the claim	Capture artifact, exact language profile, toolchain and target.
2. Identify rejection phase	lexing → parsing → static semantics → type/effect → lowering → link → load → runtime → external host.
3. Record enforcing rule	Link exact rule/diagnostic/provenance; if none exists, classify as implementation limitation.
4. Classify rationale	fundamental theory, safety, platform, resource, compatibility, convention, missing implementation, unknown.
5. Search analogues	Does another language/runtime/edition permit a semantically comparable construct?
6. Generate counterfactual	What rule/runtime/host change would make it representable?
7. Bound the experiment	Specify sandbox, resources, observables, invariants and migration constraints.
8. Build candidate	Modify the smallest required layer; do not change unrelated rules.
9. Test	Conformance, differential, property, fuzz, security, performance and regression tests.
10. Verify claimed properties	Use proofs/translation validation where feasible; otherwise label evidence strength accurately.
11. Decide	retain experimental / reject / adopt in new profile / unresolved.
12. Archive	Preserve both success and failure, exact seed, counterexamples and provenance.

Appendix C. Corelang Research Principles and Acceptance Gates

Table C1. Corelang research principles

ID	Principle
P1	Do not confuse convention with necessity.
P2	Do not confuse a compiler limitation with a language-theory limitation.
P3	Preserve failure and negative evidence.
P4	Preserve historical meaning; never silently reinterpret prior artifacts.
P5	Every rule requires rationale/provenance.
P6	Rationales are challengeable; dependencies must be tested.
P7	Novelty is not correctness.
P8	Executability is not correctness.
P9	Adaptability requires identity and version boundaries.
P10	“Impossible” must be classified by layer.
P11	Speculation must remain labeled.
P12	Abandoned designs and invalid states are research data.
P13	Probability is not proof.
P14	Test agreement is not universal semantic equivalence.
P15	No candidate promotes itself solely on its own evaluation.
P16	Unknown is a valid result.
P17	Security and capability boundaries are part of semantics, not afterthoughts.
P18	Reproducibility metadata is mandatory, not optional documentation.

22. References

[R1] HOPL. Online Historical Encyclopaedia of Programming Languages. Current roster: 8,945 languages, 7,800 influence links, 11,000+ citations. https://hopl.info/

[R2] PLDB. Programming Language DataBase repository. 5,000+ languages and 135,000+ facts. https://github.com/breck7/pldb

[R3] Yunits, B. “How Many Programming Languages are there in the World?” PLDB (2017). https://pldb.io/blog/how-many-programming-languages-are-there-in-the-world.html

[R4] ISO/IEC 9899:2024. Programming languages — C. Edition 5. International Organization for Standardization.

[R5] ISO/IEC 14882:2024. Programming languages — C++. Edition 7. International Organization for Standardization.

[R6] ISO/IEC 1539-1:2023. Programming languages — Fortran — Part 1: Base language; plus Technical Corrigendum 1:2026.

[R7] ISO/IEC 1989:2023. Programming language COBOL. International Organization for Standardization.

[R8] ISO/IEC 8652:2023. Programming languages — Ada. Edition 4.

[R9] ISO/IEC 9075-1:2023. Database languages SQL — Part 1: Framework; Technical Corrigendum 1:2026.

[R10] Ecma International. ECMA-262, 17th edition, ECMAScript 2026 Language Specification (June 2026).

[R11] Oracle. Java Language and Virtual Machine Specifications; Java SE 27 released September 2026.

[R12] Python Software Foundation. Python release documentation; PEP 602 annual release cycle; Python 3.15.0 release 9 Oct 2026.

[R13] Rust Project. The Rust Programming Language / Edition Guide: Rust 2015, 2018, 2021, and 2024 editions.

[R14] Go Project. Go 1.26 Release Notes (February 2026).

[R15] Swift.org. Swift 6.4 Released (September 2026); Swift 6.4.1 announcement (13 October 2026); Swift language revision history.

[R16] JetBrains. Kotlin release process and Kotlin language specification; Kotlin 2.4.21 released 8 October 2026.

[R17] Lua.org. Lua reference manuals; current version 5.5 and historical manuals.

[R18] Erlang/OTP. OTP 28 release and documentation; new language features in OTP 28.

[R19] Elixir. “Elixir v1.20 released: now a gradually typed language” (3 June 2026).

[R20] Julia contributors. “Julia 1.13 Highlights” (10 September 2026).

[R21] Solidity team. Solidity 0.8.36 Release Announcement (9 July 2026) and current Solidity documentation.

[R22] LLVM Project. LLVM Language Reference Manual. SSA-based common compiler intermediate representation.

[R23] LLVM Project. MLIR Language Reference and MLIR Rationale. Multi-Level Intermediate Representation.

[R24] WebAssembly Community Group. WebAssembly Core Specification 3.0 (September 2026) and specification layering.

[R25] WHATWG. HTML Living Standard, syntax and parsing sections (October 2026).

[R26] WHATWG. DOM Living Standard (October 2026).

[R27] Racket Project. Racket Guide, “Creating Languages.”

[R28] Gulwani, S., Polozov, A., & Singh, R. Program Synthesis. Foundations and Trends in Programming Languages (2017).

[R29] LLVM Project. libFuzzer documentation: coverage-guided evolutionary fuzz testing.

[R30] Leroy, X. et al. CompCert C verified compiler, version 3.18 documentation (2026).

[R31] Yang, X., Chen, Y., Eide, E., & Regehr, J. “Finding and Understanding Bugs in C Compilers.” PLDI 2011 (Csmith).

[R32] Spoofax. The Language Designer’s Workbench: syntax, static semantics, and term transformations.

[R33] NIST. SP 800-90A/B/C series on deterministic random bit generators, entropy sources, and RBG constructions.

[R34] Stanford Encyclopedia of Philosophy. “Computability and Complexity,” including the halting problem and Church–Turing thesis discussion.

[R35] Trevisan, L. Notes on Rice’s Theorem, UC Berkeley CS172 (2007).

[R36] TC39. Test262: Official ECMAScript Conformance Test Suite; 50,000+ individual test files as of May 2025.

[R37] AliveToolkit. Alive2: automatic verification / translation validation of LLVM optimizations.

[R38] Rascal. MetaProgramming documentation: programs as data for analysis and transformation.

[R39] Haskell Community. Haskell 2010 Language Report.

[R40] NIST CSRC. Random Bit Generation project and SP 800-90 publications, updated through 2026.

Research Conclusion

Corelang’s strongest preliminary thesis survives the research pass, but in a more disciplined form: language rules can be modeled as data, many can be generated and experimentally varied, and existing compiler/language-workbench/testing/proof technologies provide substantial building blocks. The thesis does not survive if interpreted as “every rule can change arbitrarily and the system will always know whether the result is correct.” The workable research program is governed evolution: explicit rule models, multi-level representations, isolated candidate execution, adversarial counterexamples, formal or empirical evidence appropriate to each claim, and immutable provenance across generations.

Under that interpretation, Corelang is not primarily a new syntax. It is an engine for asking a harder question in a testable way: which parts of a programming language are conventions, which are implementation boundaries, which are domain tradeoffs, and which are fundamental constraints—and what happens when we change exactly one of them while preserving the evidence needed to know what we actually changed?